Practical Lab - Local AI with Ollama

Running and Developing Local AI Applications
Student: Zaid Benamar
Date: 2026-10-26

1. Environment

Python 3.13.16 - Linux 6.6.159+ - 14 GB RAM - Ollama: ollama version is 0.40.11
Models: qwen3.5:4b (model 1), gemma3:4b (model 2, vision), nomic-embed-text (embeddings)

name	size_GB	family	params	quantization
nomic-embed-text:latest	0.27	nomic-bert	137M	F16
gemma3:4b	3.35	gemma3	3.9B	Q4_K_M
qwen3.5:4b	3.32	qwen35	4.2B	Q4_K_M

2. Model comparison - prime number function (Exercise 7)

Model	Response time (s)	Tokens	function	tests passed (%)	Code correctness /5	Requirements met /5	missing
qwen3.5:4b	141.3	772	is_prime	100.0	5.0	5	-
gemma3:4b	46.8	782	is_prime	100.0	5.0	5	-

- Faster: gemma3:4b (46.8 s vs 141.3 s for qwen3.5:4b). Response time depends on the model size, the number of tokens generated (782 vs 772) and on whether the model was already loaded in memory.
- Better code: both models scored the same - correctness 5.0/5 vs 5.0/5, requirements met 5/5 vs 5/5 (qwen3.5:4b vs gemma3:4b).
- More memory: gemma3:4b, which has the larger weight file (3.35 GB on disk). A loaded model uses roughly its file size in RAM/VRAM plus the context (KV cache); the exact numbers are measured in Part 16.
- For programming assistance I would choose qwen3.5:4b: it produced the most correct and complete code, even though it is not the fastest. Correct code matters more than a few seconds of latency for a coding assistant.
- Is the largest model necessarily the best? No. A bigger model has more knowledge and usually reasons better, but it needs more memory, loads more slowly and generates fewer tokens per second. A smaller model that is better trained or tuned for code can match or beat it on a given task. The choice should be based on measurements on the target machine and task, not only on parameter count.

3. Mini-RAG results (Part 13)

Q: What is reinforcement learning?
Retrieved: course.txt, reinforcement_learning.txt (1.4 s)
A: Reinforcement learning learns through interaction with an environment using rewards and penalties. An agent observes a state, chooses an action, and receives a reward. The goal is to learn a policy that maximises the cumulative reward. Q-learning and policy gradients are common algorithms.

Q: Which hardware is needed to train neural networks with many layers?
Retrieved: deep_learning.txt (0.7 s)
A: According to the context, training deep networks (which are described as having many layers) usually requires **GPUs**.

Q: What is supervised learning?
Retrieved: course.txt, machine_learning.txt (0.7 s)
A: Supervised learning uses labelled data to learn a mapping from inputs to outputs (classification, regression).

Q: Who won the 2018 World Cup?
Retrieved: (none) (0.3 s)
A: Information not available.

4. Final mini project - FSTT Local AI Assistant (Part 17)

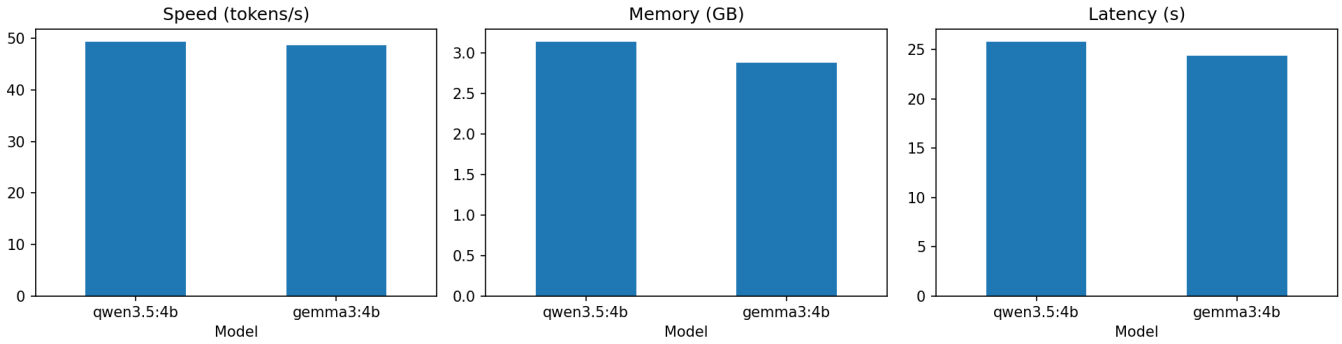
Architecture: Browser (HTML/JS) -> Flask app.py (/ask) -> TF-IDF retriever over documents/ + conversation history ->

Ollama /api/chat (localhost:11434) -> local LLM -> JSON answer with sources, response time and tokens/s.

```
                                          question  structured                                  sources  seconds
tokens/s
                    What is reinforcement learning?       False course.txt, reinforcement_learning.txt     1.43
50.1
And how is it different from supervised learning?       False       course.txt, machine_learning.txt     0.93
49.8
                             Summarize deep learning        True           deep_learning.txt, course.txt     3.00
40.9
                        What is quantum computing?       False                                            0.43
55.8
```

5. Performance benchmark (Part 16)

```
     Model  Disk size (GB)  Loaded size (GB)  VRAM (GB)  Load time (s)  Response time (s)  Tokens  Tokens/s  System
RAM used (%)  RAM increase (GB)  Quality /5  Words written
qwen3.5:4b            3.32              3.14       3.14          11.91              25.77     671     49.36
     29.4              0.49         5.0            551
 gemma3:4b            3.35              2.88       2.88           6.77              24.38     850     48.65
     27.6              0.38         5.0            598
```



On this machine (Linux, 14 GB RAM, GPU used):

- Speed: qwen3.5:4b generates 49.36 tokens/s vs 48.65 for gemma3:4b. Response time: 25.77 s (qwen3.5:4b) vs 24.38 s (gemma3:4b), including a cold load of 11.91 s and 6.77 s.
- Memory: qwen3.5:4b occupies 3.14 GB once loaded vs 2.88 GB for gemma3:4b (disk: 3.32 GB vs 3.35 GB). The loaded size is larger than the file because of the context (KV cache) and runtime buffers.
- Quality: best automatic score for qwen3.5:4b (5.0/5 on the prime-number task of Exercise 7).

Trade-off. More parameters usually means better knowledge and reasoning, but also more memory, slower loading and fewer tokens per second. On a CPU-only computer, speed is limited by memory bandwidth, so it drops roughly in proportion to model size. If the model does not fit in VRAM, part of it runs on the CPU and speed collapses. Quantization (4-bit here) is what makes 4B-parameter models fit in 3-4 GB. The right model is the smallest one that is good enough for the task on the target hardware, so measurements like these matter more than public leaderboards.

6. Answers to the report questions

Q1. What is Local AI?

Local AI means running AI models (here a large language model) directly on your own computer or server instead of calling a provider's cloud API. The model weights are downloaded once and inference is done on the local CPU/GPU, so prompts, documents and answers never leave the machine, and the model works offline.

Q2. Difference between a Cloud LLM and a Local LLM?

A cloud LLM (e.g. GPT, Claude, Gemini) runs in the provider's data centre: you send your data over the Internet to an API, pay per token, need a connection and an API key, but get very large, frontier-level models without needing hardware. A local LLM runs on your machine: data stays private, it works offline, there is no per-token cost and you control the exact model version, but you are limited to smaller models that fit your RAM/GPU, which are usually less capable and can be slower; you also manage installation and updates yourself.

Q3. What role does Ollama play?

Ollama is the local model runtime and server. It downloads and stores models from its library (ollama pull), manages them (list, show, rm), loads them into RAM/VRAM and unloads them after a timeout, runs inference with an optimised engine based on llama.cpp (CPU, NVIDIA/AMD GPU, Apple Metal), applies each model's chat template, and exposes everything through a CLI and a REST API on localhost:11434 (/api/generate, /api/chat, /api/embed, /api/tags, /api/ps...). Any program can therefore use a local model with simple HTTP requests, like a cloud API.

Q4. What is model quantization?

Quantization stores the model's weights with fewer bits per number, for example 4-bit integers (Q4_K_M) instead of 16-bit floats. A 4-billion-parameter model goes from about 8 GB in FP16 to about 2.5-3 GB in 4-bit. This reduces memory and disk usage and speeds up inference (less data to move from memory), with a small loss of accuracy. Most Ollama models are distributed in 4-bit quantization (shown as 'quantization' in Part 5).

Q5. Why does model size affect memory consumption?

To generate each token the model must use all of its weights, so all of them must be loaded in RAM or VRAM. Memory is roughly number of parameters x bytes per parameter: 4B parameters x 0.5 byte (4-bit) is about 2-2.5 GB, 9B is about 5-6 GB, plus the KV cache that grows with the context length and some runtime buffers. A larger model therefore needs more memory, and if it does not fit in the GPU it partly runs on the CPU, which is much slower.

Q6. Why can a smaller model sometimes be preferable?

A smaller model loads faster, generates more tokens per second, uses less RAM and energy, and runs on modest hardware (laptops, CPU-only machines, edge devices). For simple or narrow tasks - classification, extraction, summarising a supplied document, RAG over local files, structured JSON output - a small model is often good enough. Lower latency gives a better user experience, and several small models or many users can be served on the same machine. The best model is the smallest one that meets the quality requirement.

Q7. Advantages of local inference?

Privacy and confidentiality (data never leaves the machine - important for medical, legal or company data and GDPR); offline operation; no per-token cost or subscription; no rate limits; predictable latency without network round-trips; full control over the model version (results do not change because a provider updated the model); possibility to customise or fine-tune models; independence from a provider.

Q8. Disadvantages of local inference?

Local models are smaller and usually less capable than frontier cloud models (reasoning, knowledge, long context); performance depends on your hardware and can be slow on CPU; large models require expensive GPUs and a lot of RAM; models take several GB of disk; you must install, update and secure the software yourself; energy consumption is on your side; scaling to many users is harder than with a managed cloud service.

Q9. Explain the architecture of your Python application.

The FSTT Local AI Assistant has four layers. (1) Front-end: index.html with JavaScript sends the question with fetch() as JSON to POST /ask and displays the answer, the sources, the response time and tokens/s. (2) Back-end: a Flask app

(app.py) bound to 127.0.0.1. For each question it (3) calls the retriever (rag.py, a TF-IDF keyword retriever over the documents/ folder) to get the 2 most relevant documents, then builds the messages: a system prompt that forces answers from the context, the last 6 exchanges of the conversation history, and the context plus the question. (4) It sends them with requests to Ollama's /api/chat on localhost:11434, optionally with a JSON schema in 'format' for structured answers. Ollama runs the local LLM and returns the answer with token counts and durations; Flask stores the plain question and answer in the history, measures the time and returns JSON to the browser. chatbot.py is the console version with memory and benchmark.py measures speed and memory.

Q10. Explain the role of the REST API.

The REST API is the interface between applications and the model. Ollama exposes HTTP endpoints on localhost:11434 that accept and return JSON: /api/generate (single prompt), /api/chat (conversation with roles), /api/embed (embeddings), /api/tags and /api/ps (models on disk and in memory), /api/pull. Because it is plain HTTP + JSON, any language or tool (Python, Java, JavaScript, curl, Flask, Open WebUI) can use the model without knowing how it runs; the application and the model are decoupled, and the same code could target a remote server by changing the URL. Our own Flask app also exposes a small REST API (/ask, /reset) to the browser.

Q11. What is RAG?

Retrieval-Augmented Generation: before generating an answer, the system retrieves relevant passages from an external knowledge base (here local text files) and inserts them into the prompt as context, instructing the model to answer from that context. Pipeline: documents -> retriever (keywords/TF-IDF or embeddings with a vector index) -> relevant context -> LLM -> answer. It lets a model answer with private or up-to-date information it was never trained on, reduces hallucinations, allows citing sources, and avoids retraining the model when documents change. In Part 13, the question about the 2018 World Cup retrieved no document, so the assistant answered 'Information not available.'

Q12. Difference between an LLM and a multimodal LLM?

A standard LLM only processes text: text tokens in, text tokens out. A multimodal LLM also accepts other inputs such as images (sometimes audio or video). It adds an encoder for each modality - for images a vision transformer - and a projection layer that maps the image features into the LLM's embedding space as 'visual tokens', so the language model can reason over the image and the text together. This enables image description, visual question answering, OCR and document understanding (Part 14 used gemma3:4b, which can read images).

Q13. Compare two local models experimentally.

See the benchmark table (Part 16) and the Exercise 7 table. On this machine (Linux, 14 GB RAM, GPU used):

- Speed: qwen3.5:4b generates 49.36 tokens/s vs 48.65 for gemma3:4b. Response time: 25.77 s (qwen3.5:4b) vs 24.38 s (gemma3:4b), including a cold load of 11.91 s and 6.77 s.
- Memory: qwen3.5:4b occupies 3.14 GB once loaded vs 2.88 GB for gemma3:4b (disk: 3.32 GB vs 3.35 GB). The loaded size is larger than the file because of the context (KV cache) and runtime buffers.
- Quality: best automatic score for qwen3.5:4b (5.0/5 on the prime-number task of Exercise 7).

Trade-off. More parameters usually means better knowledge and reasoning, but also more memory, slower loading and fewer tokens per second. On a CPU-only computer, speed is limited by memory bandwidth, so it drops roughly in proportion to model size. If the model does not fit in VRAM, part of it runs on the CPU and speed collapses. Quantization (4-bit here) is what makes 4B-parameter models fit in 3-4 GB. The right model is the smallest one that is good enough for the task on the target hardware, so measurements like these matter more than public leaderboards.

Q14. Which model would you deploy on a computer with limited RAM? Explain.

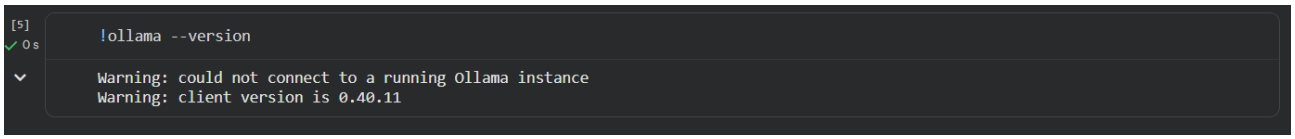
gemma3:4b, because it had the smallest memory footprint in my measurements (2.88 GB loaded) on a machine with 14 GB of RAM. On a computer with limited RAM the model must fit in memory together with the operating system and other applications; otherwise the system swaps to disk and generation becomes extremely slow, or the model cannot load at all. I would choose a small (1-4B parameter) instruct model in 4-bit quantization, keep the context window moderate (the KV cache also uses memory), load only one model at a time and set a short keep_alive so the memory is released when idle. If the quality is not sufficient, RAG over local documents is a cheaper improvement than a bigger model; with very little RAM (below 8 GB) I would move down to qwen3.5:2b or gemma3:1b.

Q15. What security risks exist when exposing a local LLM server to a network?

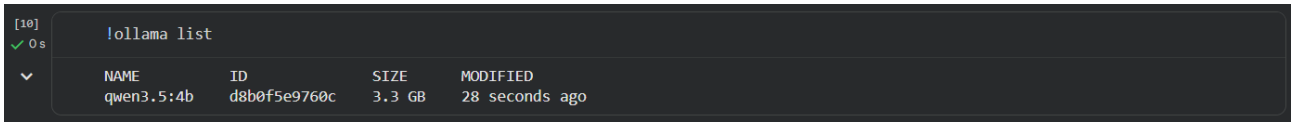
Ollama has no authentication by default. If it is exposed (OLLAMA_HOST=0.0.0.0, port 11434 open on a LAN or the Internet), anyone who can reach the port can: use your compute and electricity for free (unauthorized use, denial of service by flooding long requests); list, pull, delete or overwrite models (/api/tags, /api/pull, /api/delete), including loading malicious or unwanted models; read data sent by other users if logs or shared applications leak it; and exploit possible vulnerabilities of the server - thousands of exposed Ollama instances have been found on the Internet. At the application level there is also prompt injection: a document in the RAG folder or a user message can contain instructions that make the model ignore its system prompt or reveal other documents, and the Flask app itself has no login or rate limit. Mitigations: keep Ollama and the app bound to 127.0.0.1 (as in our app.py); if remote access is needed, put a reverse proxy (Nginx/Caddy) in front with authentication and TLS (HTTPS), restrict access with a firewall or VPN, add rate limiting and request size limits, keep Ollama updated, only use trusted models and documents, and treat model output as untrusted (never execute it automatically).

7. Screenshots

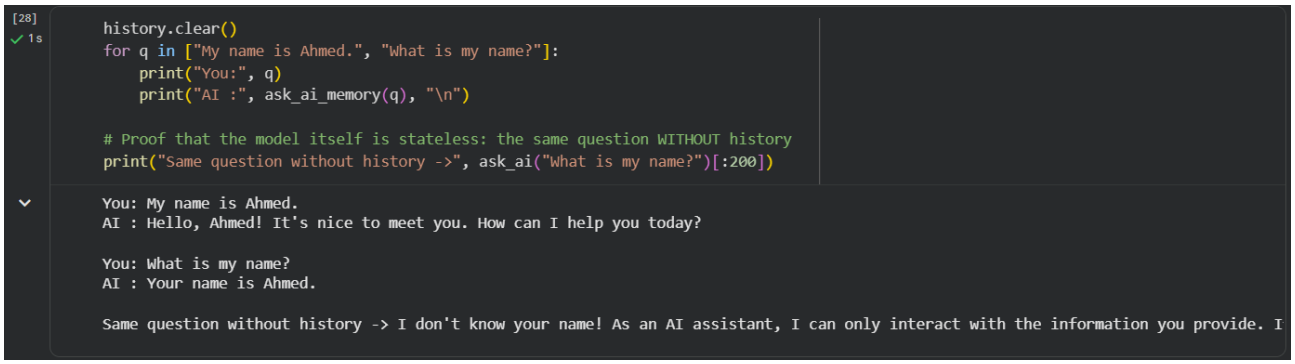
01_installation.png



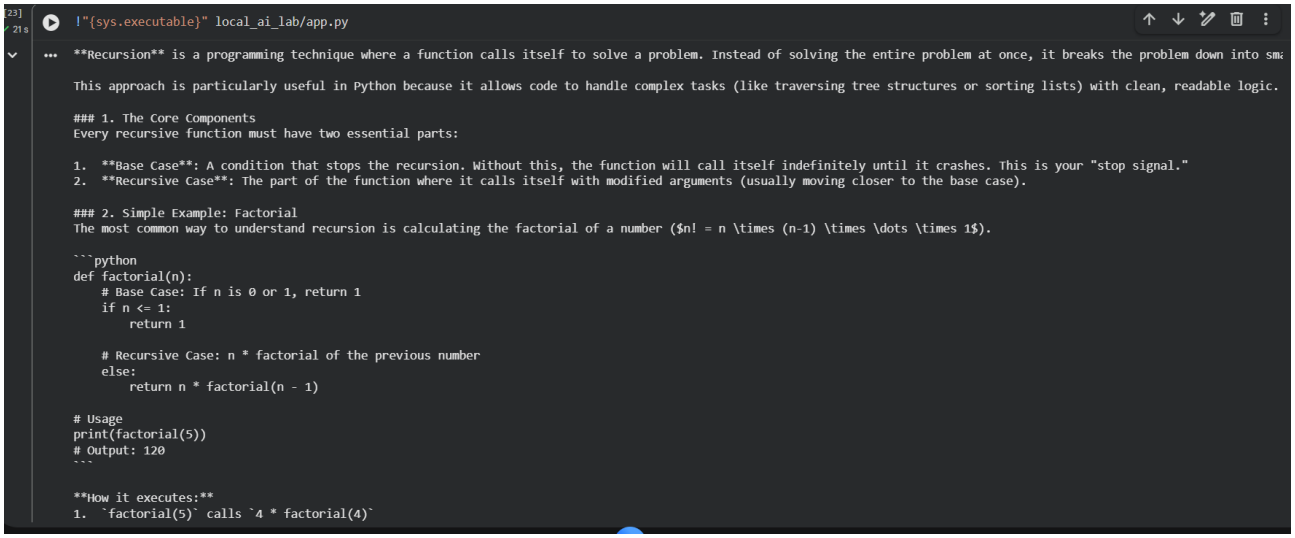
02_model.png



03_chatbot.png



04_python.png



05_rag.png

```
    return hits, out["response"], out["wall_s"]

rag_results = []
for q in ["What is reinforcement learning?",
          "Which hardware is needed to train neural networks with many layers?",
          "What is supervised learning?",
          "Who won the 2018 World Cup?"]:
    hits, ans, t = rag_answer(q)
    rag_results.append({"question": q, "retrieved": ", ".join(n for n, _ in hits) or "(none)",
                        "answer": ans.strip(), "time_s": round(t, 1)})
    print(f"Q: {q}\n  retrieved: {hits}\n  A: {ans.strip()}\n  ({t:.1f}s)\n")
```

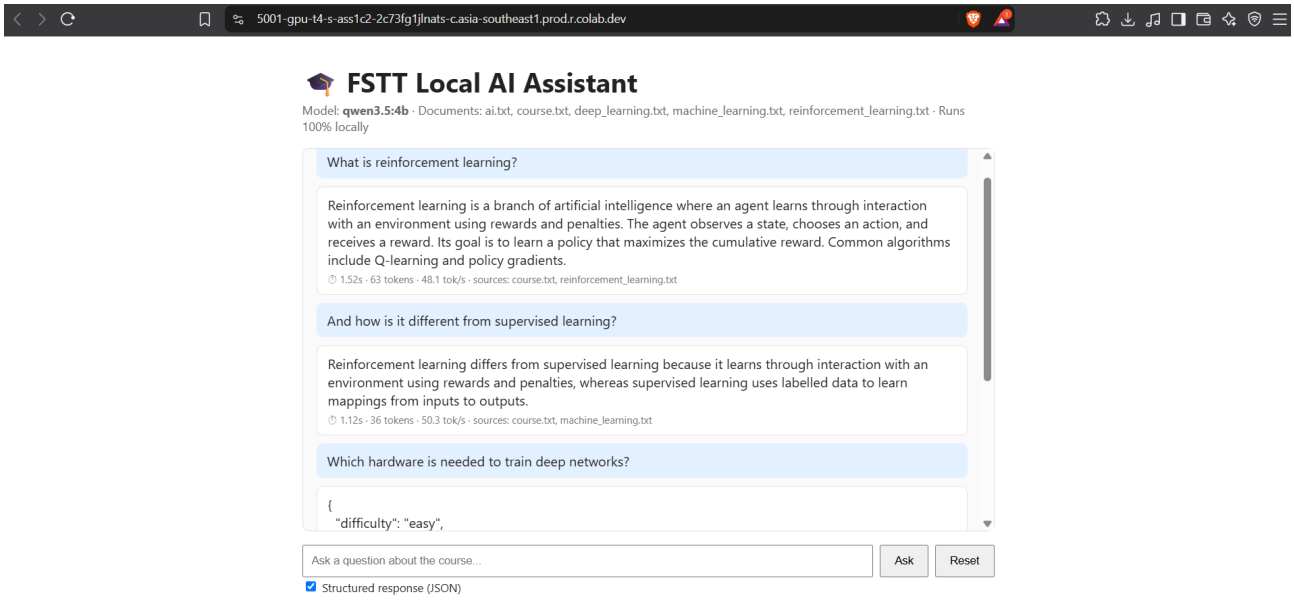
```
Q: What is reinforcement learning?
  retrieved: [('course.txt', 1.0831034956175303), ('reinforcement_learning.txt', 1.0227918049805298)]
  A: Reinforcement learning learns through interaction with an environment using rewards and penalties. An agent observes a state, chooses an action, and receives a reward. T
  (1.4s)

Q: Which hardware is needed to train neural networks with many layers?
  retrieved: [('deep_learning.txt', 8.42789552798877)]
  A: According to the context, training deep networks (which are described as having many layers) usually requires **GPUs**.
  (0.7s)

Q: What is supervised learning?
  retrieved: [('course.txt', 1.0831034956175303), ('machine_learning.txt', 1.0580718823582704)]
  A: Supervised learning uses labelled data to learn a mapping from inputs to outputs (classification, regression).
  (0.7s)

Q: Who won the 2018 World Cup?
  retrieved: []
  A: Information not available.
  (0.3s)
```

06_webapp.png



Conclusion

This lab showed that a complete AI application can run entirely on a personal computer. Ollama provides a local model server with a REST API, so the same model is usable from the terminal, Python scripts, a chatbot and a web application. Prompt design (role, context, constraints, format) strongly shapes the answers; conversation memory is implemented by the application, not by the stateless model; JSON-schema structured outputs make LLM answers reliable for software; and a simple local retriever (RAG) grounds the answers in our own documents. The benchmark showed the size trade-off: larger models need more memory and are slower, so the right local model is the smallest one that is good enough on the target hardware. Local AI offers privacy, offline use and no per-token cost, at the price of smaller models and hardware limits, and an exposed server must be secured with authentication, TLS and a firewall.